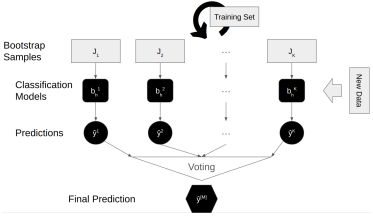
Applied Machine Learning

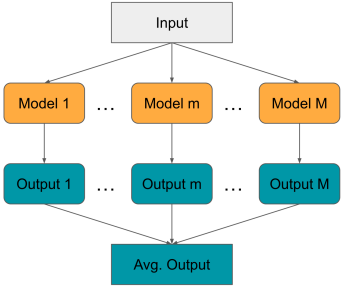
Ensembling:
Intro, Model Averaging



Learning goals

- What is Ensembling?
- Bagging and Boosting Recap
- Model Averaging

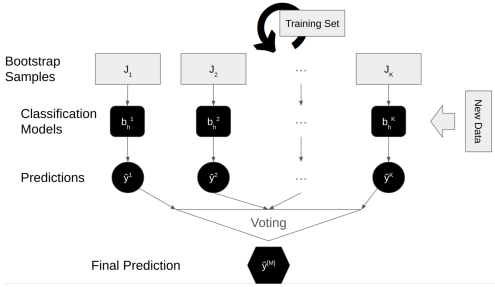
WHAT IS ENSEMBLING?



- Combine the predictions of models
- Make weak learners strong
- Make strong learners stronger
- Also: Ensembling effectively *circumvents* the problem of model selection

RECAP: BAGGING

Bootstrap **Agg**regat**ing**: Training multiple instances of the same base model on different subsets of the training data, obtained through random sampling with replacement $\rightarrow$ homogeneous & parallel ensemble



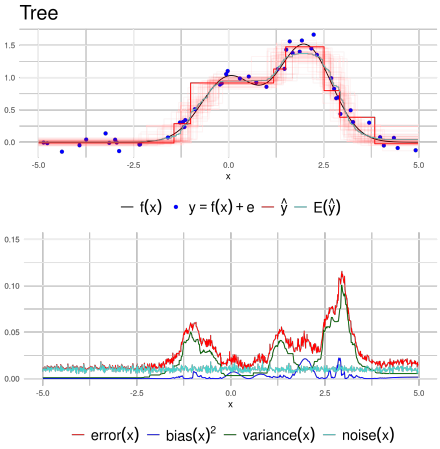
Steps:

1. Random sampling with replacement to generate different train data sets
2. Base model training
3. Combine predictions

Popular use case: Construction of RF using trees as base models

RECAP: BAGGING

Bias vs. variance: Error = Variance + Bias + Noise

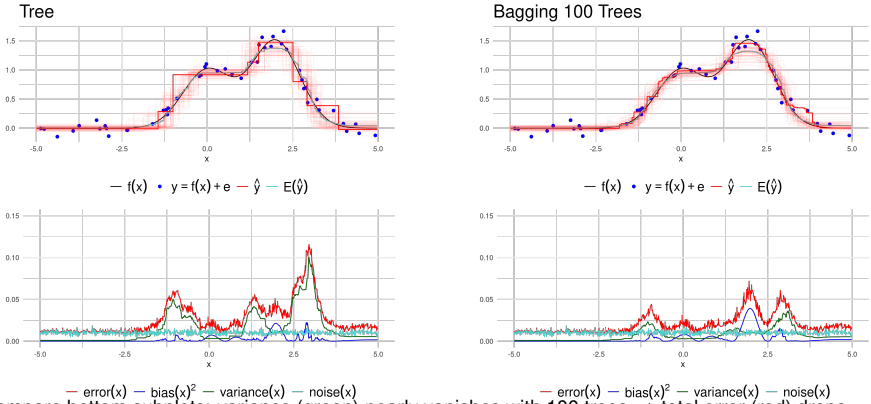


Bottom subplot shows the pointwise decomposition:

- Variance (green) is the dominant error component; bias^2 (blue) stays low
- Error peaks (red) coincide with regions of sharp change in $f(\mathbf{x})$, where individual trees vary most
- Top subplot: light red band shows the spread of individual tree fits $\hat{y}$ around $\mathbb{E}(\hat{y})$ (cyan)

RECAP: BAGGING

Bagging effect: Variance reduction through averaging; bias slightly increases (less data per model)



Compare bottom subplots: variance (green) nearly vanishes with 100 trees $\longrightarrow$ total error (red) drops dramatically. Bias (blue) stays roughly unchanged. Individual fits (light red band, top) tighten around $\mathbb{E}(\hat{y})$.

RECAP: BOOSTING

Iterative technique that sequentially trains subsequent base models to correct errors of previous models, for example, by re-weighting instances or predicting pseudo-residuals $\rightarrow$ homogeneous & sequential ensemble

Examples:

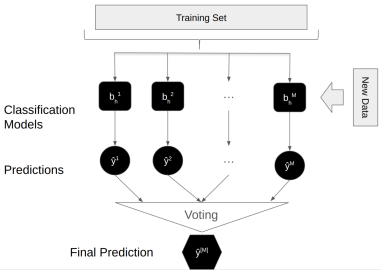
- AdaBoost (Adaptive Boosting)
- Gradient Boosting
- XGBoost (Extreme Gradient Boosting)
- LightGBM (fast and memory efficient Gradient Boosting)
- CatBoost (native support for categorical features + ordered boosting)
- Model Based Boosting

GENERAL NOTATION FOR ENSEMBLING

- $\{b^{[1]}, \ldots, b^{[M]}\} \subseteq \mathcal{H}_1 \times \ldots \times \mathcal{H}_M$ set of M models trained on training data $\mathcal{D}$
- $\pi^{[m]} : \mathcal{X} \to [0,1]^g$ probability predictions under the m-th model when doing classification (w.l.o.g. we assume $g = 1$ in the following and $\pi^{[m]}$ returning the probabilities of the positive class)
- $h^{[m]} : \mathcal{X} \to \mathcal{Y}$ hard label predictions under the m-th model when doing classification or regression
- $f^{[M]}(\mathbf{x}) = G(b^{[1]}(\mathbf{x}), \ldots, b^{[M]}(\mathbf{x}))$ prediction of an ensemble model obtained via aggregating function G

MODEL AVERAGING

Combine predictions of multiple (different) models to make a final prediction → heterogeneous



Steps:

1. Base model training
2. Combine predictions

Classification: majority voting (hard voting) / model averaging (soft voting)

- $f^{[M]}(\mathbf{x}) = \text{mode}(\{h^{[1]}(\mathbf{x}), \ldots, h^{[M]}(\mathbf{x})\})$ assuming hard label predictions
- $f^{[M]}(\mathbf{x}) = \frac{1}{M}\sum_{m=1}^{M} \pi^{[m]}(\mathbf{x})$ assuming probability predictions

Regression: model averaging

- $f^{[M]}(\mathbf{x}) = \frac{1}{M}\sum_{m=1}^{M} b^{[m]}(\mathbf{x})$

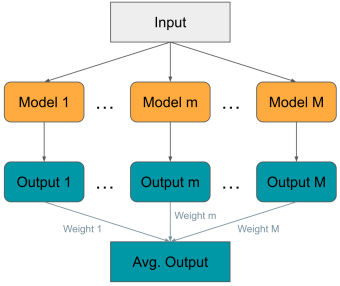
WEIGHTED MODEL AVERAGING

Assume a set of M different models obtained from data $\mathcal{D}$ using a resampling $\mathcal{J}$.

Assign weights $\mathbf{w} = (w_1, \ldots, w_M)$, usually: $0 \leq w_m \leq 1, \sum_{m=1}^{M} w_m = 1$ to the models

Classification: $f_{\mathbf{w}}(\mathbf{x}) = \sum_{m=1}^{M} w_m \cdot \pi^{[m]}(\mathbf{x})$

Regression: $f_{\mathbf{w}}(\mathbf{x}) = \sum_{m=1}^{M} w_m \cdot b^{[m]}(\mathbf{x})$



EXAMPLE: WEIGHTED MODEL AVERAGING

Balanced accuracy (BACC) on the sonar task, using a stratified holdout split (1/3 test set):

Learner	**BACC** (Test)
RF	0.834
k-NN (k=7)	0.792
Log. Reg.	0.695
Model Averaging (Equal Weights)	0.796
Model Averaging ($\mathbf{w} = (0.75, 0.05, 0.20)^\intercal$)	0.810

How can we find 'good' weights?

NB: Model averaging with equal weights will not result in better performance out of the box; depends on types of learners, their diversity in predictions, ...

WEIGHTED MODEL AVERAGING CONTINUED

Find the optimal weight vector $\mathbf{w}^\star$ such that the estimated generalization error (based on a performance metric ρ or loss function L) of the ensemble is optimal:

$$\mathbf{w}^\star := \arg\min_{\mathbf{w} \in \mathbb{R}^M} \hat{\mathrm{GE}}\left(\hat{f}_{\mathbf{w}}, \mathcal{J}, \rho\right)$$

Performance metric/loss function unknown a priori (e.g., as in AutoML) $\rightarrow$ black box optimization problem.

Difficulty of the optimization problem depends on the metric/loss function:

- MSE (regression) $L_{\mathrm{MSE}}(y^{(i)}, \hat{f}_{\mathbf{w}}(\mathbf{x}^{(i)})) = (y^{(i)} - \hat{f}_{\mathbf{w}}(\mathbf{x}^{(i)}))^2$;
  Constraints of the form $0 \leq w_m \leq 1, \sum_{m=1}^{M} w_m = 1$;

  $$\arg\min_{\mathbf{w} \in \mathbb{R}^M} \hat{\mathrm{GE}}(\hat{f}_{\mathbf{w}}, \mathcal{J}, L_{\mathrm{MSE}}) \text{ s.t. } 0 \leq w_m \leq 1, \sum_{m=1}^{M} w_m = 1$$

  can be written as a quadratic program.
- 0-1 loss (binary classification); the optimization problem is NP-hard.

In practice, we use black box optimization, greedy ensemble selection, or a linear model (stacking).

GREEDY ENSEMBLE SELECTION

Assume a set of M different models obtained from data $\mathcal{D}$ using a resampling $\mathcal{J}$. We now want to optimize the weights used in weighted model averaging.

Greedy ensemble selection (GES, see ▸ "Caruana et al." 2004) is one of the most popular methods to do so:

1. Start with the empty ensemble
2. Add the base model that maximizes the ensemble's performance metric on a hillclimb (validation) set
3. Repeat step 2 until a given number of iterations are reached
4. Return the ensemble from the set of generated ensembles that has the best performance on the validation set

NB: GES optimizes the weights on an integer domain (i.e., base models are selected one or multiple times or not at all[1].)

[1] This has the (positive) side-effect of resulting in a sparse weight vector

EXAMPLE: GES

Iter	Sel. Learners	Current BACC (Valid)	BACC if Learner added (Valid)		
			RF	k-NN	Log. Reg.
1	{RF: 0, k-NN: 0, Log. Reg.: 0}	NA	**0.837**	0.809	0.766
2	{RF: 1, k-NN: 0, Log. Reg.: 0}	0.837	**0.837**	0.809	0.766
3	{RF: 2, k-NN: 0, Log. Reg.: 0}	0.837	**0.837**	0.809	0.809
4	{RF: 3, k-NN: 0, Log. Reg.: 0}	0.837	**0.837**	0.769	0.832
5	{RF: 4, k-NN: 0, Log. Reg.: 0}	0.837	0.837	0.792	**0.855**
6	{RF: 4, k-NN: 0, Log. Reg.: 1}	0.855	0.897	**0.917**	0.809
7	{RF: 4, k-NN: 1, Log. Reg.: 1}	**0.917**	**0.917**	0.895	0.809
8	{RF: 5, k-NN: 1, Log. Reg.: 1}	0.917	**0.897**	0.875	0.875
9	{RF: 6, k-NN: 1, Log. Reg.: 1}	0.897	**0.877**	0.875	0.875
10	{RF: 7, k-NN: 1, Log. Reg.: 1}	0.877	0.857	0.855	**0.917**
11	{RF: 7, k-NN: 1, Log. Reg.: 2}	0.917	...	...	...

Best iteration: 6 with BACC (Valid) of 0.917

$\rightarrow$ {RF: 4, k-NN: 1, Log. Reg.: 1}

$\rightarrow \mathbf{w}^\star = (\frac{4}{6}, \frac{1}{6}, \frac{1}{6})^\intercal$

EXAMPLE: CONCLUSION

Learner	**BACC** (Valid)	**BACC** (Test)
RF	0.837	0.834
k-NN (k=7)	0.809	0.792
Log. Reg.	0.766	0.695
Model Averaging (Equal Weights)	0.832	0.796
Model Averaging ($\mathbf{w} = (0.75, 0.05, 0.20)^\intercal$)	0.875	0.810
Model Averaging (GES: $\mathbf{w}^\star = (\frac{4}{6}, \frac{1}{6}, \frac{1}{6})^\intercal$)	0.917	0.864

NB: Optimizing weights can (and often does) overfit to the validation data; especially if standard black box optimizers are used

CLOSING

Why does ensembling work? ▸ "Dietterich" 2000

- **Statistical Problem:** Limited data → many models achieve similar training error; averaging reduces the risk of picking a bad one
- **Computational Problem:** Optimization can get stuck in local optima; combining multiple runs explores more of the hypothesis space
- **Representational Problem:** The true function may lie outside any single hypothesis space; a combination can better approximate it

Requirements to build an ensemble:

- **Diverse Models:** Base models should make different errors (complementary strengths)
  - Use different algorithms
  - Use different hyperparameter configurations of the same algorithm
  - Use different feature subsets or data subsets
- **Strong Models:** Individual models should perform reasonably well

Practical Suggestion: Greedy Ensemble Selection works really well in practice.